SCP - External Secrets

- CyberArk and the Current State
- External Secrets Operator
  - Architecture
  - Authentication
- How To
  - Onboard your safe
  - Usage Examples

CyberArk and the Current State

The combined bank (FCB/SVB) uses CyberArk as our secret store. Very simply this means that CyberArk is the source of truth for secret data including application passwords, license keys and the like. The CyberArk platform run by the Identity and Access Management (IAM) team is a separate architecture and requires integration to work in a kubernetes environment.

The Vendored solution provided by CyberArk is called Conjur. The Container Platforms team has attempted several times to encourage the adoption of Conjur but also acknowledge the significant cost and complexity of such a system.

In addition to Conjur, CyberArk exposes an API endpoint called Central Credential Provider (CCP). CCP can be used to pull credentials using automation but doesn't offer the same level of Authz controls as a full solution such as Conjur, however we may be able to solve these concerns at a lower level.

External Secrets Operator

The External Secrets Operator is an open source controller that can be integrated into kubernetes. In addition to supporting CyberArk Conjur officially, the External Secrets Operator can be setup to connect with any secrets provider using a generic webhook.

Architecture

To align with CyberArk's method of one Vault per Application team, we intend to deploy ESO-As-A-Service.

Each team has their own vault, and within our deployment model will get their own SecretStore that is configured to talk only to that vault.

blocked URL

Authentication

CyberArk CCP offers different auth methods:

- AppID - A Unique ID tied to a vault. The CyberArk/IAM Support team create this, and this is a required configuration on the SecretStore object.
- IP Whitelisting - Typically a bad way to handle security, however, given the "As-A-Service" deployment strategy, we can restrict connections to CCP from a select group of **Infrastructure Nodes** within each cluster where ESO is running.
- Active Directory user/password - CCP can be configured to use an active directory service account with basic auth (Username:Password). This can be configured within ESO and tied directly to a secretstore. IAM supports one user/password for ESO in general.

How To

Onboard your safe

The Container Platforms team has setup a ServiceNow form (Container Platforms: Onboard Namespace to CyberArk) to request onboarding of your safes. Safe onboarding will create a **ClusterSecretStore** object matching the safe name (the only difference is the underscores will be changed to dashes to comply with RFC1123) . You will need to know this name in order to sync secrets into a cluster. Please see the Usage Examples below

Usage Examples

ESO can be as complex as you need it to be, from fetching just the password and writing that field, to templating an entire secret and injecting the secret data into the appropriate place.

Please see the upstream documentation or the LSVB wiki for more examples.

```
           apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: example
  namespace: my-namespace
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: namespace-secretstore-example
    kind: ClusterSecretStore
  target:
    name: kube-secret-to-be-created
    creationPolicy: Owner
  data:
  - secretKey: kube-secret-key-to-be-managed
    remoteRef:
      key: cyberark-username-field
      property: cyberark-address-field
```

Creates a **secret** that looks like so:

```
           apiVersion: v1
kind: Secret
metadata:
  annotations:
    reconcile.external-secrets.io/data-hash: ...
  labels:
    reconcile.external-secrets.io/created-by: ...
  name: kube-secret-to-be-created
  ownerReferences:
  - apiVersion: external-secrets.io/v1beta1
    blockOwnerDeletion: true
    controller: true
    kind: ExternalSecret
    name: example
data:
  kube-secret-key-to-be-managed: bmljZSB0cnksIG1heWJlIG5leHQgdGltZQ==
immutable: false
type: Opaque
```

```
           apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: template-example
spec:
  refreshInterval: "30m"
  secretStoreRef:
    name: namespace-secretstore-example
    kind: ClusterSecretStore
  target:
    name: kube-secret-to-be-created
    # this is how the Kind=Secret will look like
    template:
      data:
        config.properties: |
          spring.option.config.something=true
          spring.option.config.another=string
          spring.whatever.config.profile=dev

          spring.auth.config.username=myusername
          spring.auth.config.password={{ .password }}
          spring.service.2.username=svc.myuser
          spring.service.2.password={{ .service2password }}

          config.property.url=https://my-example.com
          config.property.username=svc.myuser
          config.property.password={{ .service2password }} # Example
showing how to reference the same secret multiple times

  data:
  - secretKey: password
    remoteRef:
      key: <cyberark-username-field-for-password>
      property: cyberark-address-field
  - secretKey: service2password
    remoteRef:
      key: <cyberark-username-field-for-other-password>
      property: cyberark-address-field
```

Creates a secret that looks like so:

```
           apiVersion: v1
kind: Secret
metadata:
  annotations:
    reconcile.external-secrets.io/data-hash: ...
  labels:
    reconcile.external-secrets.io/created-by: ...
  name: kube-secret-to-be-created
  ownerReferences:
  - apiVersion: external-secrets.io/v1beta1
    blockOwnerDeletion: true
    controller: true
    kind: ExternalSecret
    name: template-example
data:
  config.properties:
c3ByaW5nLm9wdGlvbi5jb25maWcuc29tZXRoaW5nPXRydWUKc3ByaW5nLm9wdGlvbi5jb25maWc
uYW5vdGhlcj1zdHJpbmcKc3ByaW5nLndoYXRldmVyLmNvbmZpZy5wcm9maWxlPWRldgoKc3ByaW
5nLmF1dGguY29uZmlnLnVzZXJuYW1lPW15dXNlcm5hbWUKc3ByaW5nLmF1dGguY29uZmlnLnBhc
3N3b3JkPW5vdHJlYWwKc3ByaW5nLnNlcnZpY2UuMi51c2VybmFtZT1zdmMubXl1c2VyCnNwcmlu
Zy5zZXJ2aWNlLjIucGFzc3dvcmQ9bmljZXRyeQoKY29uZmlnLnByb3BlcnR5LnVybD1odHRwczo
vL215LWV4YW1wbGUuY29tCmNvbmZpZy5wcm9wZXJ0eS51c2VybmFtZT1zdmMubXl1c2VyCmNvbm
ZpZy5wcm9wZXJ0eS5wYXNzd29yZD1uaWNldHJ5Cg==
immutable: false
type: Opaque
```

```
           apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: template-example
spec:
  refreshInterval: "30m"
  secretStoreRef:
    name: namespace-secretstore-example
    kind: ClusterSecretStore
  target:
    name: kube-secret-to-be-created
    # this is how the Kind=Secret will look like
    template:
      data:
        engineVersion: v2
        # multiline string
        config: |
          datasources:
          - name: Graphite
            type: graphite
            access: proxy
            url: http://hard-coded:8080
            password: "{{ .password }}"
            user: "{{ .user }}"
        # using replace function to rewrite secret
        connection: '{{ .dburl | replace "postgres://" "postgresql://" }}'

  data:
  - secretKey: user
    remoteRef:
      key: <cyberark-username-field-for-user>
      property: cyberark-address-field
  - secretKey: password
    remoteRef:
      key: <cyberark-username-field-for-password>
      property: cyberark-address-field
  - secretKey: dburl
    remoteRef:
      key: <cyberark-username-field-for-dburl>
      property: cyberark-address-field
```

Creates a secret that looks like so:

```
           apiVersion: v1
kind: Secret
metadata:
  annotations:
    reconcile.external-secrets.io/data-hash: ...
  labels:
    reconcile.external-secrets.io/created-by: ...
  name: kube-secret-to-be-created
  ownerReferences:
  - apiVersion: external-secrets.io/v1beta1
    blockOwnerDeletion: true
    controller: true
    kind: ExternalSecret
    name: template-example
data:
  config:
ZGF0YXNvdXJjZXM6CiAgLSBuYW1lOiBHcmFwaGl0ZQogICAgdHlwZTogZ3JhcGhpdGUKICAgIGF
jY2VzczogcHJveHkKICAgIHVybDogaHR0cDovL2hhcmQtY29kZWQ6ODA4MAogICAgcGFzc3dvcm
Q6ICJjM1J2Y0NCaVpXbHVaeUJqYkdWMlpYSXNJSEJzWldGelpRPT0iCiAgICB1c2VyOiAibXktd
XNlcm5hbWUiCg==
  connection: dGhpcyBpcyBhbiBleGFtcGxlLCByZW1lbWJlcj8=
  engineVersion: aGFyZGNvZGVkLWtleQ==
immutable: false
type: Opaque
```

Templates can store golang templates as well, allowing dynamic rendering of the end secret based on the context. This also allows using common golang sprig functions, such:

- `b64enc` or `b64dec` to deal with base64 encoding
- `now` to print the timestamp
- `sha256sum` to produce a checksum

```
           ---
apiVersion: v1
kind: ConfigMap
metadata:
  name: grafana-config-tpl
data:
  config.yaml: |
    datasources:
      - name: Graphite
        type: graphite
        access: proxy
        url: "{{ .uri }}"
        password: "{{ .password }}"
        user: "{{ .user }}"
  templated: |
     # key and value both templated
     my-application-{{ .user }}: {{ .password | b64enc }}
     # Optional values
     {{- if hasPrefix "oci://" .uri }}
     enableOCI: true
     {{- else }}
     enableOCI: false
     {{- end }}
     application-type: grafana
  annotations: |
     #dynamic timestamp generation using the `now` function
     last-synced-for-user/{{ .user }}: {{ now }}
---
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: my-template-example
spec:
  refreshInterval: "30m"
  secretStoreRef:
```

```
    name: namespace-secretstore-example
    kind: ClusterSecretStore
  target:
    name: secret-to-be-created
    template:
      engineVersion: v2
      templateFrom:
      # Target the portion of the secret to manage
      - target: Data
        configMap:
          # name of the configmap to pull in
          name: grafana-config-tpl
          # here you define the keys that should be used as template
          items:
          - key: config.yaml
            templateAs: Values
          - key: templated
            templateAs: KeysAndValues
      # Target the portion of the secret to manage
      - target: Annotations
        configMap:
          # name of the configmap to pull in
          name: grafana-config-tpl
          # here you define the keys that should be used as template
          items:
          - key: annotations
            templateAs: KeysAndValues
  data:
  - secretKey: user # These keys are the important bit used in the template
    remoteRef:
      key: eso-test-username
      property: uat.svbank.com
  - secretKey: password
    remoteRef:
      key: eso-test-password
      property: uat.svbank.com
      decodingStrategy: Auto  # Because the value is base64 encoded inside
cyberark
  - secretKey: uri
    remoteRef:
      key: eso-test-url
      property: uat.svbank.com
      decodingStrategy: None  # Because the value is clear-text inside
cyberark
```

Creates a secret that looks like so:

```
           apiVersion: v1
kind: Secret
metadata:
  annotations:
    last-synced-for-user/test-user: 2024-04-05 17:35:27.958280071 +0000
UTC m=+87508.114829878
    reconcile.external-secrets.io/data-hash: ...
  labels:
    reconcile.external-secrets.io/created-by: ...
  name: secret-to-be-created
  ownerReferences:
  - apiVersion: external-secrets.io/v1beta1
    blockOwnerDeletion: true
    controller: true
    kind: ExternalSecret
    name: my-template-example
data:
  application-type: Z3JhZmFuYQ==
  config.yaml:
ZGF0YXNvdXJjZXM6CiAgLSBuYW1lOiBHcmFwaGl0ZQogICAgdHlwZTogZ3JhcGhpdGUKICAgIGF
jY2VzczogcHJveHkKICAgIHVybDogIm9jaTovL3RoaXNpc2F0ZXN0OjgwODAiCiAgICBwYXNzd2
9yZDogIlRoSXNJczRwNHNTdzByRCEiCiAgICB1c2VyOiAidGVzdC11c2VyIgo=
  enableOCI: dHJ1ZQ==
  my-application-test-user: VkdoSmMwbHpOSEEwYzFOM01ISkVJUT09
immutable: false
type: Opaque
```